MoA Prediction — Hyperparameter Tuning, drug-aware (hybrid CV)

Version of `moa-tuning-row.ipynb` (tuning v2) aligned with the `CV_SCHEME = "hybrid"` protocol of the training notebook.

Key Changes Compared to the Row-Wise Version

- External folds **and** internal split (early stopping / scheduler) built using the exact same functions as the training notebook: `make_split_groups` + `grouped_multilabel_splits` . Drugs with more than `HYBRID_MIN_ROWS` rows are split row-by-row, while less frequent drugs are kept intact within a single fold.
- Tuning folds use a `SPLIT_SEED` different from the training seed (42), so they are a different partition from the CV folds of the training notebook. They are not disjoint from them: rows used for tuning also appear in the validation folds of the final CV (see Limitations in the main README).
- Confirmation on `CONFIRM_FOLD` evaluated across **multiple seeds** (`CONFIRM_SEEDS`), with paired evaluations between default and candidate configurations. Decisions are made strictly on the confirmation fold (since the search fold is biased by selecting the best among trials): requires an average gain $\geq \mathrm{MIN_GAIN}$ **and** victories on more than half of the seeds.
- Search spaces expanded at the boundaries where optimal values previously settled (ResNet dropout/hidden/batch, FT lr/dropout, CatBoost depth/l2). Pruner configured with a longer warm-up phase (MedianPruner tends to penalize slow-converging configurations, which are typical when using high dropout rates).
- Output: `best_hyperparameters_hybrid.json` (same format read by the training pipeline) and `tuning_report.json` containing protocol details, seed configurations, and per-seed results.

Requires `train_drug.csv` (in the competition directory or as notebook input) and `HYBRID_MIN_ROWS` matching the training pipeline setup.

In [1]:
```
# Step 1 - Install Dependencies
!pip install -q iterative-stratification
```

In [2]:
```
# Step 2 - Imports and Global Configuration
import os
import gc
import json
import glob
import time
import math
import traceback
import warnings
import contextlib
```

```python
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import optuna
from sklearn.decomposition import PCA
from sklearn.preprocessing import QuantileTransformer
from sklearn.feature_selection import VarianceThreshold
from sklearn.exceptions import DataConversionWarning
from iterstrat.ml_stratifiers import MultilabelStratifiedKFold, MultilabelStratifiedShuffleSplit
from catboost import CatBoostClassifier

warnings.filterwarnings("ignore", category=DataConversionWarning)
warnings.filterwarnings("ignore")
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Constants and Parameters
SMOKE_TEST = False               # If True, runs a fast test (~5 min) on a reduced subset
DATA_DIR = "/kaggle/input/competitions/lish-moa/"
OUTPUT_DIR = "/kaggle/working"
SEED = 42
RANDOM_STATE = 42
CTL_LABEL = "ctl_vehicle"
CLIP_EPS = 1e-15

# Folds & Two-stage evaluation
N_FOLDS = 5
SEARCH_FOLD = 0                 # Fold for Stage 1 (search)
CONFIRM_FOLD = 1                # Fold for Stage 2 (confirmation)

# Drug-aware protocol (must match the training notebook)
CV_SCHEME = "hybrid"
HYBRID_MIN_ROWS = 18             # drugs with MORE rows than this are split by row
DRUG_FILE = "train_drug.csv"
SPLIT_SEED = 1234              # different from the training seed (42) on purpose
CONFIRM_SEEDS = [42, 202, 2024] # seeds used in Stage 2, identical for default and candidates

# Preprocessing & Feature Engineering
N_QUANTILES = 100
PCA_G_COMPONENTS = 50
PCA_C_COMPONENTS = 15
VARIANCE_THRESHOLD = 0.0

# Training Recipe
INNER_VAL_FRACTION = 0.1
LABEL_SMOOTHING = 0.001
```

```
PLATEAU_FACTOR, PLATEAU_PATIENCE = 0.1, 3
RESNET_MAX_EPOCHS, RESNET_PATIENCE = 35, 8
FT_MAX_EPOCHS, FT_PATIENCE = 35, 8
EVAL_BATCH_SIZE = {"resnet": 1024, "ft": 128}
CATBOOST_ITERATIONS = 250

# Tuning Parameters (Number of search trials replacing time limits)
N_TRIALS_SEARCH = {"resnet": 25, "catboost": 15, "ft": 15}
FT_SEARCH_ARCHITECTURE = False
K_CONFIRM = 2
MIN_GAIN = 1e-4
MIN_TRIALS_FOR_A_SEARCH = 8

if SMOKE_TEST:
    N_ROWS_SMOKE = 4000
    N_TRIALS_SEARCH = {"resnet": 3, "catboost": 2, "ft": 2}
    RESNET_MAX_EPOCHS, FT_MAX_EPOCHS = 3, 2
    RESNET_PATIENCE, FT_PATIENCE = 2, 1
    CATBOOST_ITERATIONS = 20
    K_CONFIRM = 1
    CONFIRM_SEEDS = [42, 202]
    MIN_TRIALS_FOR_A_SEARCH = 2

N_GPUS = torch.cuda.device_count()
DEVICE = torch.device("cuda:0" if N_GPUS > 0 else "cpu")
os.makedirs(OUTPUT_DIR, exist_ok=True)
print(f"Device in use: {DEVICE} | Available GPUs: {N_GPUS} | Smoke Test: {SMOKE_TEST}")
```

```
Device in use: cuda:0 | Available GPUs: 2 | Smoke Test: False
```

In [3]:

```
# Step 3 - Data Loading, drug ids and drug-aware fold pre-computation
def find_drug_file(data_dir):
    path = os.path.join(data_dir, DRUG_FILE)
    if os.path.exists(path):
        return path
    matches = glob.glob(f"/kaggle/input/**/{DRUG_FILE}", recursive=True)
    if len(matches) == 1:
        return matches[0]
    raise FileNotFoundError(f"'{DRUG_FILE}' not found in {data_dir} ({len(matches)} matches under /kaggle/input)")


def load_and_preprocess_data(data_dir=DATA_DIR):
    if not os.path.exists(os.path.join(data_dir, "train_features.csv")):
        matches = glob.glob(f"/kaggle/input/**/train_features.csv", recursive=True)
        if len(matches) > 0:
            data_dir = os.path.dirname(matches[0]) + "/"

    train_features = pd.read_csv(f"{data_dir}train_features.csv")
```

```
    train_targets = pd.read_csv(f"{data_dir}train_targets_scored.csv")
    assert (train_features["sig_id"].values == train_targets["sig_id"].values).all(), "features/targets not aligned"

    drug_table = pd.read_csv(find_drug_file(data_dir))
    assert {"sig_id", "drug_id"} <= set(drug_table.columns) and drug_table["sig_id"].is_unique
    drug_map = drug_table.set_index("sig_id")["drug_id"]

    # Remove control samples (ctl_vehicle)
    non_control_mask = train_features["cp_type"] != CTL_LABEL
    train_features = train_features[non_control_mask].reset_index(drop=True)
    train_targets = train_targets[non_control_mask].reset_index(drop=True)
    drug_ids = train_features["sig_id"].map(drug_map)
    assert drug_ids.notna().all(), "some sig_id have no drug_id"
    drug_ids = drug_ids.astype(str).values

    feature_cols = [c for c in train_features.columns if c not in ["sig_id", "cp_type"]]
    target_cols = [c for c in train_targets.columns if c != "sig_id"]
    g_cols = [c for c in feature_cols if c.startswith("g-")]
    c_cols = [c for c in feature_cols if c.startswith("c-")]

    # Row-wise statistics for g- and c- features
    df_out = train_features[[c for c in train_features.columns if c != "cp_type"]].copy()
    df_out["g_sum"] = df_out[g_cols].sum(axis=1)
    df_out["g_mean"] = df_out[g_cols].mean(axis=1)
    df_out["g_std"] = df_out[g_cols].std(axis=1)
    df_out["g_kurtosis"] = df_out[g_cols].kurtosis(axis=1)
    df_out["g_skew"] = df_out[g_cols].skew(axis=1)

    df_out["c_sum"] = df_out[c_cols].sum(axis=1)
    df_out["c_mean"] = df_out[c_cols].mean(axis=1)
    df_out["c_std"] = df_out[c_cols].std(axis=1)
    df_out["c_kurtosis"] = df_out[c_cols].kurtosis(axis=1)
    df_out["c_skew"] = df_out[c_cols].skew(axis=1)

    dummy_cols = ["cp_time", "cp_dose"]
    df_out = pd.get_dummies(df_out, columns=dummy_cols, dtype=np.float64)

    full_feature_cols = [c for c in df_out.columns if c != "sig_id"]
    dummy_feature_cols = [c for c in full_feature_cols if any(c.startswith(d) for d in dummy_cols)]

    X = df_out[full_feature_cols].values.astype(np.float64)
    Y = train_targets[target_cols].values

    if SMOKE_TEST:
        keep = np.sort(np.random.default_rng(SEED).choice(len(X), size=min(N_ROWS_SMOKE, len(X)), replace=False))
        X, Y, drug_ids = X[keep], Y[keep], drug_ids[keep]

    return X, Y, drug_ids, full_feature_cols, g_cols, c_cols, dummy_feature_cols
```

```python
# --- split logic copied from the training notebook (Step 2.5) ---
def grouped_multilabel_splits(Y, groups, n_splits, seed, first_only=False):
    unique_groups, group_of_row = np.unique(groups, return_inverse=True)
    group_labels = np.zeros((len(unique_groups), Y.shape[1]), dtype=np.float64)
    np.maximum.at(group_labels, group_of_row, Y)
    splitter = MultilabelStratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    splits = []
    for _, val_groups in splitter.split(np.zeros((len(unique_groups), 1)), group_labels):
        is_val = np.isin(group_of_row, val_groups)
        train_idx, val_idx = np.where(~is_val)[0], np.where(is_val)[0]
        assert set(groups[train_idx]).isdisjoint(groups[val_idx]), "a group is split across folds"
        splits.append((train_idx, val_idx))
        if first_only:
            break
    return splits


def make_hybrid_groups(drug_ids, min_rows):
    _, drug_of_row, rows_per_drug = np.unique(drug_ids, return_inverse=True, return_counts=True)
    is_frequent = rows_per_drug[drug_of_row] > min_rows
    row_groups = np.char.add("row_", np.arange(len(drug_ids)).astype(str))
    assert not set(row_groups) & set(drug_ids), "row-group labels collide with drug ids"
    groups = np.where(is_frequent, row_groups, drug_ids)
    print(f"Hybrid groups: {int(is_frequent.sum())} rows of {len(np.unique(drug_ids[is_frequent]))} frequent drugs "
          f"(> {min_rows} rows) split by row; {len(np.unique(drug_ids[~is_frequent]))} other drugs kept whole")
    return groups


def prepare_cached_folds(X, Y, groups, drug_ids, feature_cols, g_cols, c_cols, dummy_cols):
    """Pre-computes PCA and Raw transformations on the drug-aware folds. Outer folds and the inner
    split use the same grouping rule as the training notebook (CV_SCHEME = hybrid)."""
    print(f"Pre-computing drug-aware folds (seed {SPLIT_SEED})...")
    splits = grouped_multilabel_splits(Y, groups, N_FOLDS, SPLIT_SEED)
    sizes = [len(v) for _, v in splits]
    print(f"Validation rows per fold: {sizes} | rows whose drug is also in the training fold: "
          + ", ".join(f"{np.isin(drug_ids[v], drug_ids[tr]).mean():.1%}" for tr, v in splits))
    cached_folds = {}
    as32 = lambda arr: np.ascontiguousarray(arr, dtype=np.float32)

    for fold_id, (train_idx, val_idx) in enumerate(splits):
        if fold_id not in (SEARCH_FOLD, CONFIRM_FOLD):
            continue
        X_train, X_val = X[train_idx], X[val_idx]
        Y_train, Y_val = Y[train_idx], Y[val_idx]

        selector = VarianceThreshold(threshold=VARIANCE_THRESHOLD)
```

```
        X_train_vt = selector.fit_transform(X_train)
        X_val_vt = selector.transform(X_val)
        support = selector.get_support()
        selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

        dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
        cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]
        cont_cols = [selected_cols[i] for i in cont_idx]
        g_idx = [i for i, c in enumerate(cont_cols) if c in g_cols]
        c_idx = [i for i, c in enumerate(cont_cols) if c in c_cols]

        qt = QuantileTransformer(n_quantiles=N_QUANTILES, output_distribution="normal", random_state=RANDOM_STATE)
        tf_train = qt.fit_transform(X_train_vt[:, cont_idx])
        tf_val = qt.transform(X_val_vt[:, cont_idx])

        pca_g = PCA(n_components=PCA_G_COMPONENTS, random_state=RANDOM_STATE)
        g_train = pca_g.fit_transform(tf_train[:, g_idx])
        g_val = pca_g.transform(tf_val[:, g_idx])

        pca_c = PCA(n_components=PCA_C_COMPONENTS, random_state=RANDOM_STATE)
        c_train = pca_c.fit_transform(tf_train[:, c_idx])
        c_val = pca_c.transform(tf_val[:, c_idx])

        dummy_train, dummy_val = X_train_vt[:, dummy_idx], X_val_vt[:, dummy_idx]

        # Inner split for early stopping / scheduler: grouped like the outer one (same rule as make_inner_split)
        fit_idx, inner_idx = grouped_multilabel_splits(
            Y_train, groups[train_idx], n_splits=max(2, round(1 / INNER_VAL_FRACTION)),
            seed=SEED, first_only=True)[0]
        assert set(groups[train_idx][fit_idx]).isdisjoint(groups[train_idx][inner_idx])

        cached_folds[fold_id] = {
            "fold": fold_id,
            "nn_train": as32(np.hstack([tf_train, g_train, c_train, dummy_train])),
            "nn_val": as32(np.hstack([tf_val, g_val, c_val, dummy_val])),
            "raw_train": as32(np.hstack([tf_train, dummy_train])),
            "raw_val": as32(np.hstack([tf_val, dummy_val])),
            "Y_train": Y_train, "Y_val": Y_val,
            "fit_idx": fit_idx, "inner_idx": inner_idx
        }
    return cached_folds
```

In [4]:

```
# Step 4 - Models (TabularResNet, FTTransformer)
class TabularResNet(nn.Module):
    name = "resnet"
    def __init__(self, input_dim, output_dim=206, hidden_dim=512, dropout_rate=0.2):
        super().__init__()
        self.in_proj = nn.Sequential(
```

```
            nn.BatchNorm1d(input_dim),
            nn.Dropout(dropout_rate),
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
        )
        self.block1 = self._make_block(hidden_dim, dropout_rate)
        self.block2 = self._make_block(hidden_dim, dropout_rate)
        self.out_proj = nn.Sequential(
            nn.SiLU(), nn.Dropout(dropout_rate), nn.Linear(hidden_dim, output_dim)
        )

    @staticmethod
    def _make_block(hidden_dim, dropout_rate):
        return nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
        )

    def forward(self, x):
        x = self.in_proj(x)
        x = F.silu(x + self.block1(x))
        x = F.silu(x + self.block2(x))
        return self.out_proj(x)


class FeatureTokenizer(nn.Module):
    def __init__(self, num_features, d_token):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(num_features, d_token))
        self.bias = nn.Parameter(torch.empty(num_features, d_token))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        nn.init.zeros_(self.bias)

    def forward(self, x):
        return x.unsqueeze(-1) * self.weight + self.bias


class FTTransformer(nn.Module):
    name = "ft_transformer"
    def __init__(self, input_dim, output_dim=206, d_token=64, n_blocks=2,
                 attention_heads=4, dropout=0.2):
        super().__init__()
```

```
        self.tokenizer = FeatureTokenizer(input_dim, d_token)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_token))
        nn.init.normal_(self.cls_token, std=0.02)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token, nhead=attention_heads, dim_feedforward=d_token * 4,
            dropout=dropout, activation="gelu", batch_first=True,
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_blocks, enable_nested_tensor=False)
        self.layer_norm = nn.LayerNorm(d_token)
        self.head = nn.Sequential(nn.Dropout(dropout), nn.Linear(d_token, output_dim))

    def forward(self, x):
        tokens = self.tokenizer(x)
        cls_tokens = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat((cls_tokens, tokens), dim=1)
        x = self.transformer(x)
        cls_out = self.layer_norm(x[:, 0, :])
        return self.head(cls_out)
```

In [5]:

```
# Step 5 - Training and Evaluation Functions
def log_loss_float64(y_true, probabilities, eps=CLIP_EPS):
    p = np.clip(np.asarray(probabilities, dtype=np.float64), eps, 1 - eps)
    y = np.asarray(y_true, dtype=np.float64)
    return float(np.mean(-(y * np.log(p) + (1 - y) * np.log(1 - p))))


@torch.no_grad()
def predict_probabilities(model, X, batch_size, use_amp):
    model.eval()
    chunks = []
    for start in range(0, X.shape[0], batch_size):
        with torch.autocast(device_type=X.device.type, dtype=torch.float16, enabled=use_amp):
            logits = model(X[start:start + batch_size])
        chunks.append(torch.sigmoid(logits.float()).cpu().numpy())
    return np.vstack(chunks).astype(np.float64)


def make_gpu_data(train_array, val_array, fold, device):
    X_train = torch.as_tensor(train_array, dtype=torch.float32, device=device)
    y_train = torch.as_tensor(fold["Y_train"], dtype=torch.float32, device=device)
    fit_idx = torch.as_tensor(fold["fit_idx"], dtype=torch.long, device=device)
    inner_idx = torch.as_tensor(fold["inner_idx"], dtype=torch.long, device=device)
    return {
        "X_fit": X_train[fit_idx], "y_fit": y_train[fit_idx],
        "X_inner": X_train[inner_idx], "y_inner": fold["Y_train"][fold["inner_idx"]],
        "X_val": torch.as_tensor(val_array, dtype=torch.float32, device=device), "y_val": fold["Y_val"]
    }
```

```python
def train_and_score(model, data, *, lr, weight_decay, batch_size, max_epochs, patience, eval_batch_size,
                    device, trial=None, seed=SEED):
    use_amp = device.type == "cuda"
    model = model.to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=PLATEAU_FACTOR, patience=PLATEAU_PATIENCE)
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

    X_fit, y_fit = data["X_fit"], data["y_fit"]
    y_smoothed = y_fit * (1 - LABEL_SMOOTHING) + 0.5 * LABEL_SMOOTHING
    n_rows = X_fit.shape[0]
    steps_per_epoch = max(1, n_rows // batch_size)
    generator = torch.Generator(device=device)
    generator.manual_seed(seed)

    best_inner_loss, best_state, epochs_without_improvement, epochs_run = float("inf"), None, 0, 0
    for epoch in range(max_epochs):
        model.train()
        order = torch.randperm(n_rows, device=device, generator=generator)
        for step in range(steps_per_epoch):
            batch = order[step * batch_size:(step + 1) * batch_size]
            optimizer.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                logits = model(X_fit[batch])
            loss = criterion(logits.float(), y_smoothed[batch])
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

        inner_loss = log_loss_float64(data["y_inner"], predict_probabilities(model, data["X_inner"], eval_batch_size, use_amp))
        if not np.isfinite(inner_loss):
            raise optuna.TrialPruned()
        scheduler.step(inner_loss)
        epochs_run += 1
        if inner_loss < best_inner_loss:
            best_inner_loss = inner_loss
            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
            epochs_without_improvement = 0
        else:
            epochs_without_improvement += 1
        if trial is not None:
            trial.report(inner_loss, epoch)
            if trial.should_prune():
                raise optuna.TrialPruned()
        if epochs_without_improvement >= patience:
            break
```

```
    if best_state is None:
        raise optuna.TrialPruned()
    model.load_state_dict(best_state)
    val_loss = log_loss_float64(data["y_val"], predict_probabilities(model, data["X_val"], eval_batch_size, use_amp))
    return {"val_loss": val_loss, "best_inner_loss": float(best_inner_loss), "epochs": epochs_run}


def run_guarded(function, *args, **kwargs):
    try:
        return function(*args, **kwargs)
    except torch.cuda.OutOfMemoryError:
        print("  CUDA out of memory: trial pruned")
        raise optuna.TrialPruned()
    finally:
        gc.collect()
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
```

In [6]:

```
# Step 6 - Search Spaces and Model Evaluators
RESNET_DEFAULT = {"lr": 1e-3, "dropout": 0.2, "weight_decay": 1e-5, "hidden_dim": 512, "batch_size": 256}
FT_DEFAULT = {"lr": 1e-3, "dropout": 0.2, "weight_decay": 1e-5, "d_token": 64, "n_blocks": 2, "attention_heads": 4, "batch_size": 64}
CATBOOST_DEFAULT = {"depth": 5, "l2_leaf_reg": 3.0, "learning_rate": 0.08, "subsample": None, "iterations": CATBOOST_ITERATIONS}
DEFAULT_CONFIG = {"resnet": RESNET_DEFAULT, "ft": FT_DEFAULT, "catboost": CATBOOST_DEFAULT}

DEFAULT_SUGGESTED = {
    "resnet": dict(RESNET_DEFAULT),
    "ft": {k: FT_DEFAULT[k] for k in ("lr", "dropout", "weight_decay", "batch_size")
           + (("d_token", "n_blocks", "attention_heads") if FT_SEARCH_ARCHITECTURE else ())},
    "catboost": {"depth": 5, "l2_leaf_reg": 3.0, "learning_rate": 0.08, "bootstrap": "default"},
}

def suggest_resnet(trial):
    return {
        "lr": trial.suggest_float("lr", 3e-4, 3e-3, log=True),
        "dropout": trial.suggest_float("dropout", 0.1, 0.6),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True),
        "hidden_dim": trial.suggest_categorical("hidden_dim", [256, 512, 768, 1024]),
        "batch_size": trial.suggest_categorical("batch_size", [64, 128, 256, 512])
    }

def suggest_ft(trial):
    params = {
        "lr": trial.suggest_float("lr", 3e-4, 6e-3, log=True),
        "dropout": trial.suggest_float("dropout", 0.0, 0.35),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-3, log=True),
        "batch_size": trial.suggest_categorical("batch_size", [64, 128])
    }
```

```
    if FT_SEARCH_ARCHITECTURE:
        params.update(
            d_token=trial.suggest_categorical("d_token", [32, 64]),
            n_blocks=trial.suggest_categorical("n_blocks", [1, 2]),
            attention_heads=trial.suggest_categorical("attention_heads", [2, 4])
        )
    return params

def suggest_catboost(trial):
    params = {
        "depth": trial.suggest_int("depth", 3, 6),
        "l2_leaf_reg": trial.suggest_float("l2_leaf_reg", 1.0, 60.0, log=True),
        "learning_rate": trial.suggest_float("learning_rate", 0.04, 0.25, log=True),
        "bootstrap": trial.suggest_categorical("bootstrap", ["default", "Bernoulli"])
    }
    if params["bootstrap"] == "Bernoulli":
        params["subsample"] = trial.suggest_float("subsample", 0.6, 0.95)
    return params

SUGGEST = {"resnet": suggest_resnet, "ft": suggest_ft, "catboost": suggest_catboost}

def build_config(kind, params):
    if kind in ("resnet", "ft"):
        base = DEFAULT_CONFIG[kind]
        return {**base, **{key: value for key, value in params.items() if key in base}}
    config = dict(CATBOOST_DEFAULT)
    for key in ("depth", "l2_leaf_reg", "learning_rate"):
        if key in params:
            config[key] = params[key]
    config["subsample"] = params.get("subsample") if params.get("bootstrap") == "Bernoulli" else None
    return config

def evaluate_resnet(config, fold, device, trial=None, seed=SEED):
    data = make_gpu_data(fold["nn_train"], fold["nn_val"], fold, device)
    n_inputs, n_outputs = data["X_fit"].shape[1], fold["Y_train"].shape[1]
    torch.manual_seed(seed)
    model = TabularResNet(n_inputs, n_outputs, hidden_dim=config["hidden_dim"], dropout_rate=config["dropout"])
    return train_and_score(model, data, lr=config["lr"], weight_decay=config["weight_decay"],
                           batch_size=config["batch_size"], max_epochs=RESNET_MAX_EPOCHS, patience=RESNET_PATIENCE,
                           eval_batch_size=EVAL_BATCH_SIZE["resnet"], device=device, trial=trial, seed=seed)

def evaluate_ft(config, fold, device, trial=None, seed=SEED):
    data = make_gpu_data(fold["raw_train"], fold["raw_val"], fold, device)
    n_inputs, n_outputs = data["X_fit"].shape[1], fold["Y_train"].shape[1]
    torch.manual_seed(seed)
    model = FTTransformer(n_inputs, n_outputs, d_token=config["d_token"], n_blocks=config["n_blocks"],
                          attention_heads=config["attention_heads"], dropout=config["dropout"])
    return train_and_score(model, data, lr=config["lr"], weight_decay=config["weight_decay"],
```

```
                           batch_size=config["batch_size"], max_epochs=FT_MAX_EPOCHS, patience=FT_PATIENCE,
                           eval_batch_size=EVAL_BATCH_SIZE["ft"], device=device, trial=trial, seed=seed)

def evaluate_catboost(config, fold, device, trial=None, seed=SEED):
    Y_train, Y_val = fold["Y_train"], fold["Y_val"]
    is_constant = Y_train.min(axis=0) == Y_train.max(axis=0)
    active, constant = np.where(~is_constant)[0], np.where(is_constant)[0]
    prior = (Y_train.sum(axis=0) + 0.5) / (Y_train.shape[0] + 1.0)
    params = dict(loss_function="MultiLogloss", iterations=config["iterations"], depth=config["depth"],
                  learning_rate=config["learning_rate"], l2_leaf_reg=config["l2_leaf_reg"],
                  random_seed=seed, verbose=False, allow_writing_files=False)
    if config["subsample"] is not None:
        params.update(bootstrap_type="Bernoulli", subsample=config["subsample"])
    if N_GPUS > 0:
        params.update(task_type="GPU", devices="0")
    model = CatBoostClassifier(**params)
    model.fit(fold["raw_train"], Y_train[:, active])
    predictions = np.zeros(Y_val.shape, dtype=np.float64)
    predictions[:, active] = model.predict_proba(fold["raw_val"])
    if constant.size:
        predictions[:, constant] = prior[constant]
    return {"val_loss": log_loss_float64(Y_val, predictions), "best_inner_loss": None, "epochs": None}

EVALUATE = {"resnet": evaluate_resnet, "ft": evaluate_ft, "catboost": evaluate_catboost}

def make_objective(kind, fold, device, durations):
    def objective(trial):
        params = SUGGEST[kind](trial)
        config = build_config(kind, params)
        start = time.time()
        result = run_guarded(EVALUATE[kind], config, fold, device, trial)
        seconds = time.time() - start
        durations.append(seconds)
        trial.set_user_attr("seconds", round(seconds, 1))
        if result.get("epochs") is not None:
            trial.set_user_attr("epochs", result["epochs"])
            trial.set_user_attr("best_inner_loss", result["best_inner_loss"])
        return result["val_loss"]
    return objective
```

In [7]:

```
# Step 7 - Stage 2 and Decision Rule (Unrestricted Evaluation)
LABEL = {"resnet": "ResNet", "ft": "FTTransformer", "catboost": "CatBoost"}
N_STARTUP = {"resnet": 10, "ft": 4, "catboost": 4}
PRUNER_FACTORY = {
    "resnet": lambda: optuna.pruners.MedianPruner(n_startup_trials=8, n_warmup_steps=10),
    "ft": lambda: optuna.pruners.MedianPruner(n_startup_trials=3, n_warmup_steps=10),
    "catboost": lambda: optuna.pruners.NopPruner()
}
```

```
COMPLETE = optuna.trial.TrialState.COMPLETE

def to_jsonable(obj):
    if isinstance(obj, dict):
        return {str(key): to_jsonable(value) for key, value in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [to_jsonable(value) for value in obj]
    if isinstance(obj, np.generic):
        return obj.item()
    return obj

def make_callback(kind):
    def callback(study, trial):
        value = "-" if trial.value is None else f"{trial.value:.5f}"
        try:
            best = f"{study.best_value:.5f}"
        except ValueError:
            best = "-"
        epochs = f", {trial.user_attrs['epochs']} epochs" if "epochs" in trial.user_attrs else ""
        print(f"[{kind}] trial {trial.number:02d} {trial.state.name:8s} loss={value} "
              f"({trial.user_attrs.get('seconds', 0):.0f}s{epochs}) best={best}")
        recent = study.trials[-3:]
        if len(recent) == 3 and all(t.state == optuna.trial.TrialState.FAIL for t in recent):
            print(f"[{kind}] three consecutive failed trials: stopping search")
            study.stop()
    return callback

def run_search(kind, fold, device):
    study = optuna.create_study(direction="minimize", study_name=kind,
                                sampler=optuna.samplers.TPESampler(seed=SEED, n_startup_trials=N_STARTUP[kind]),
                                pruner=PRUNER_FACTORY[kind]())
    study.enqueue_trial(DEFAULT_SUGGESTED[kind])
    durations = []
    n_trials = N_TRIALS_SEARCH[kind]
    print(f"[{kind}] Running search for {n_trials} trials...")
    study.optimize(make_objective(kind, fold, device, durations),
                   n_trials=n_trials, callbacks=[make_callback(kind)],
                   gc_after_trial=True, catch=(Exception,))
    return study

def confirm_candidates(kind, study, fold, device):
    """Stage 2: top candidates + default re-evaluated on the CONFIRMATION fold with the same seeds for all."""
    complete = sorted([t for t in study.trials if t.state == COMPLETE], key=lambda t: t.value)
    if not complete:
        return []
    default = next((t for t in complete if t.number == 0), None)
    ordered = []
    for trial in [complete[0], default, *complete[1:K_CONFIRM]]:
```

```
        if trial is not None and all(trial.number != other.number for other in ordered):
            ordered.append(trial)
    rows = []
    for trial in ordered:
        row = {"trial": trial.number, "is_default": trial.number == 0, "params": dict(trial.params),
               "config": build_config(kind, trial.params), "score_search": float(trial.value),
               "score_confirm": None, "confirm_by_seed": [], "note": ""}
        try:
            for seed in CONFIRM_SEEDS:
                result = run_guarded(EVALUATE[kind], row["config"], fold, device, None, seed)
                row["confirm_by_seed"].append(float(result["val_loss"]))
            row["score_confirm"] = float(np.mean(row["confirm_by_seed"]))
        except optuna.TrialPruned:
            row["note"] = "trial pruned"; row["score_confirm"] = None
        except Exception:
            row["note"] = "error: " + traceback.format_exc().strip().splitlines()[-1]; row["score_confirm"] = None
        confirm = "-" if row["score_confirm"] is None else f"{row['score_confirm']:.5f} (seed std {np.std(row['confirm_by_seed']):.5f})"
        print(f"[{kind}] confirmation trial {row['trial']:02d}{' (default)' if row['is_default'] else ''}: "
              f"search fold {row['score_search']:.5f} | confirmation fold {confirm} {row['note']}")
        rows.append(row)
    return rows

def decide(kind, rows):
    """Adopt a candidate only if, on the confirmation fold and averaged over seeds, it beats the default by
    at least MIN_GAIN AND wins on more than half of the (paired) seeds. The search-fold score is reported
    but not used: it is the minimum over N trials and therefore biased."""
    decision = {"adopted": False, "config": dict(DEFAULT_CONFIG[kind]), "reason": "", "default_search": None,
                "default_confirm": None, "best_trial": None, "best_search": None, "best_confirm": None,
                "mean_gain": None, "relative_gain": None, "seed_wins": None, "n_seeds": len(CONFIRM_SEEDS),
                "default_seed_std": None}
    default = next((r for r in rows if r["is_default"]), None)
    if default is not None:
        decision.update(default_search=default["score_search"], default_confirm=default["score_confirm"],
                        default_seed_std=float(np.std(default["confirm_by_seed"])) if default["confirm_by_seed"] else None)
    if default is None or default["score_confirm"] is None:
        decision["reason"] = "default was not evaluated on the confirmation fold, keeping default"
        return decision
    candidates = [r for r in rows if not r["is_default"] and r["score_confirm"] is not None]
    if not candidates:
        decision["reason"] = "no candidate confirmed on the confirmation fold, keeping default"
        return decision
    best = min(candidates, key=lambda r: r["score_confirm"])
    gain = default["score_confirm"] - best["score_confirm"]
    wins = int(sum(b < d for b, d in zip(best["confirm_by_seed"], default["confirm_by_seed"])))
    decision.update(best_trial=best["trial"], best_search=best["score_search"], best_confirm=best["score_confirm"],
                    mean_gain=gain, relative_gain=gain / default["score_confirm"], seed_wins=wins)
    if gain >= MIN_GAIN and wins > len(CONFIRM_SEEDS) / 2:
        decision.update(adopted=True, config=best["config"],
```

```
                        reason=f"better than default on the confirmation fold: gain {gain:.5f} >= MIN_GAIN {MIN_GAIN}, "
                               f"wins {wins}/{len(CONFIRM_SEEDS)} seeds")
    else:
        reasons = []
        if gain < MIN_GAIN:
            reasons.append(f"mean gain {gain:+.5f} < MIN_GAIN {MIN_GAIN}")
        if wins <= len(CONFIRM_SEEDS) / 2:
            reasons.append(f"wins only {wins}/{len(CONFIRM_SEEDS)} seeds")
        decision["reason"] = "keeping default: " + "; ".join(reasons)
    return decision

def write_outputs(results, errors, started, complete):
    suffix = "_SMOKE_TEST" if SMOKE_TEST else ""
    final = {LABEL[kind]: (results[kind]["decision"]["config"] if kind in results else dict(DEFAULT_CONFIG[kind]))
             for kind in LABEL}
    report = {"complete": complete, "smoke_test": SMOKE_TEST, "elapsed_minutes": round((time.time() - started) / 60, 1),
              "protocol": {"cv_scheme": CV_SCHEME, "hybrid_min_rows": HYBRID_MIN_ROWS, "split_seed": SPLIT_SEED, "confirm_seeds": CONFIRM_SEEDS,
"search_fold": SEARCH_FOLD, "confirm_fold": CONFIRM_FOLD, "seed": SEED,
                           "min_gain": MIN_GAIN, "k_confirm": K_CONFIRM, "n_trials_search": N_TRIALS_SEARCH},
              "models": {LABEL[kind]: to_jsonable(value) for kind, value in results.items()},
              "errors": errors}
    with open(f"{OUTPUT_DIR}/best_hyperparameters_hybrid{suffix}.json", "w") as f:
        json.dump(to_jsonable(final), f, indent=4)
    with open(f"{OUTPUT_DIR}/tuning_report{suffix}.json", "w") as f:
        json.dump(to_jsonable(report), f, indent=2)

def print_summary(results):
    rows = []
    for kind in LABEL:
        if kind not in results:
            rows.append({"model": LABEL[kind], "decision": "NOT TUNED"})
            continue
        d, counts = results[kind]["decision"], results[kind]["counts"]
        fmt = lambda v: "-" if v is None else f"{v:.5f}"
        rows.append({"model": LABEL[kind], "trials (complete/pruned/failed)":
                     f"{counts.get('COMPLETE', 0)}/{counts.get('PRUNED', 0)}/{counts.get('FAIL', 0)}",
                     "default (search/confirm)": f"{fmt(d['default_search'])} / {fmt(d['default_confirm'])}",
                     "best (search/confirm)": f"{fmt(d['best_search'])} / {fmt(d['best_confirm'])}", "seed wins": "-" if d["seed_wins"] is None else
f"{d['seed_wins']}/{d['n_seeds']}",
                     "mean gain": "-" if d["mean_gain"] is None else f"{d['mean_gain']:+.5f} ({d['relative_gain']:+.1%})",
                     "adopted": "TUNED" if d["adopted"] else "default"})
    print(pd.DataFrame(rows).to_string(index=False))
    for kind in results:
        print(f"  {LABEL[kind]}: {results[kind]['decision']['reason']}")
```

In [8]:

```
# Step 8 - Sequential Execution Pipeline (Stable on GPU T4)
def run_tuning_pipeline():
    started = time.time()
```

```
print("=== STARTING HYPERPARAMETER TUNING PIPELINE (ResNet, CatBoost, FT-Transformer) ===")

# 1. Data Loading
X, Y, drug_ids, feature_cols, g_cols, c_cols, dummy_cols = load_and_preprocess_data()
groups = make_hybrid_groups(drug_ids, HYBRID_MIN_ROWS)
print(f"Dataset Loaded -> Shape X: {X.shape}, Shape Y: {Y.shape}")

# 2. Pre-computing Fold Transformations
folds = prepare_cached_folds(X, Y, groups, drug_ids, feature_cols, g_cols, c_cols, dummy_cols)
print(f"Inputs Ready in RAM after {(time.time() - started) / 60:.1f} min | "
      f"ResNet input: {folds[SEARCH_FOLD]['nn_train'].shape[1]} columns, "
      f"Raw input: {folds[SEARCH_FOLD]['raw_train'].shape[1]} columns")

results, errors = {}, {}
models_order = ["resnet", "catboost", "ft"]

# Sequential execution to prevent VRAM/CUDA conflicts in Jupyter
for kind in models_order:
    print(f"\n--- STARTING TUNING FOR {LABEL[kind].upper()} ---")
    try:
        # Stage 1: Search
        study = run_search(kind, folds[SEARCH_FOLD], DEVICE)
        counts = {}
        for trial in study.trials:
            counts[trial.state.name] = counts.get(trial.state.name, 0) + 1

        try:
            study.trials_dataframe().to_csv(f"{OUTPUT_DIR}/trials_{kind}{'_SMOKE_TEST' if SMOKE_TEST else ''}.csv", index=False)
        except Exception:
            pass

        if counts.get("COMPLETE", 0) < MIN_TRIALS_FOR_A_SEARCH:
            print(f"[{kind}] WARNING: only {counts.get('COMPLETE', 0)} completed trials (< {MIN_TRIALS_FOR_A_SEARCH})")

        # Stage 2: Confirmation
        rows = confirm_candidates(kind, study, folds[CONFIRM_FOLD], DEVICE)
        decision = decide(kind, rows)

        results[kind] = {
            "decision": decision, "counts": counts, "candidates": rows,
            "finished_after_min": round((time.time() - started) / 60, 1)
        }
        write_outputs(results, errors, started, complete=False)
        print(f"[{kind}] DECISION: {decision['reason']}")

    except Exception:
        errors[kind] = traceback.format_exc()
        print(f"[{kind}] ERROR, default value kept:\n{errors[kind]}")
```

```
        finally:
            gc.collect()
            if torch.cuda.is_available():
                torch.cuda.empty_cache()

    write_outputs(results, errors, started, complete=True)
    print(f"\n=== COMPLETED in {(time.time() - started) / 60:.1f} min ===")
    print_summary(results)
    return results
```

In [9]:

```
# Step 9 - Pipeline Execution
RESULTS = run_tuning_pipeline()
print()
suffix = "_SMOKE_TEST" if SMOKE_TEST else ""
print(open(f"{OUTPUT_DIR}/best_hyperparameters_hybrid{suffix}.json").read())
```

```
=== STARTING HYPERPARAMETER TUNING PIPELINE (ResNet, CatBoost, FT-Transformer) ===
Hybrid groups: 2142 rows of 9 frequent drugs (> 18 rows) split by row; 3279 other drugs kept whole
Dataset Loaded -> Shape X: (21948, 887), Shape Y: (21948, 206)
Pre-computing drug-aware folds (seed 1234)...
Validation rows per fold: [4308, 4380, 4409, 4450, 4401] | rows whose drug is also in the training fold: 9.9%, 9.8%, 9.7%, 9.5%, 9.9%
Inputs Ready in RAM after 0.5 min | ResNet input: 952 columns, Raw input: 887 columns

--- STARTING TUNING FOR RESNET ---
[resnet] Running search for 25 trials...
[resnet] trial 00 COMPLETE loss=0.01867 (12s, 13 epochs) best=0.01867
[resnet] trial 01 COMPLETE loss=0.01852 (35s, 30 epochs) best=0.01852
[resnet] trial 02 COMPLETE loss=0.01867 (6s, 20 epochs) best=0.01852
[resnet] trial 03 COMPLETE loss=0.01846 (13s, 21 epochs) best=0.01846
[resnet] trial 04 COMPLETE loss=0.01828 (6s, 35 epochs) best=0.01828
[resnet] trial 05 COMPLETE loss=0.01872 (9s, 15 epochs) best=0.01828
[resnet] trial 06 COMPLETE loss=0.01913 (13s, 11 epochs) best=0.01828
[resnet] trial 07 COMPLETE loss=0.01835 (37s, 32 epochs) best=0.01828
[resnet] trial 08 PRUNED   loss=0.01914 (0s) best=0.01828
[resnet] trial 09 COMPLETE loss=0.01888 (8s, 13 epochs) best=0.01828
[resnet] trial 10 COMPLETE loss=0.01841 (4s, 25 epochs) best=0.01828
[resnet] trial 11 PRUNED   loss=0.01944 (0s) best=0.01828
[resnet] trial 12 COMPLETE loss=0.01852 (24s, 21 epochs) best=0.01828
[resnet] trial 13 COMPLETE loss=0.01872 (4s, 27 epochs) best=0.01828
[resnet] trial 14 COMPLETE loss=0.01849 (20s, 17 epochs) best=0.01828
[resnet] trial 15 PRUNED   loss=0.01890 (0s) best=0.01828
[resnet] trial 16 PRUNED   loss=0.01817 (0s) best=0.01828
[resnet] trial 17 PRUNED   loss=0.01901 (0s) best=0.01828
[resnet] trial 18 COMPLETE loss=0.01852 (23s, 20 epochs) best=0.01828
[resnet] trial 19 PRUNED   loss=0.01784 (0s) best=0.01828
[resnet] trial 20 PRUNED   loss=0.01832 (0s) best=0.01828
[resnet] trial 21 PRUNED   loss=0.01818 (0s) best=0.01828
[resnet] trial 22 COMPLETE loss=0.01843 (3s, 20 epochs) best=0.01828
[resnet] trial 23 COMPLETE loss=0.01846 (3s, 18 epochs) best=0.01828
[resnet] trial 24 PRUNED   loss=0.01816 (0s) best=0.01828
[resnet] confirmation trial 04: search fold 0.01828 | confirmation fold 0.01748 (seed std 0.00005)
[resnet] confirmation trial 00 (default): search fold 0.01867 | confirmation fold 0.01780 (seed std 0.00002)
[resnet] confirmation trial 07: search fold 0.01835 | confirmation fold 0.01748 (seed std 0.00005)
[resnet] DECISION: better than default on the confirmation fold: gain 0.00032 >= MIN_GAIN 0.0001, wins 3/3 seeds

--- STARTING TUNING FOR CATBOOST ---
[catboost] Running search for 15 trials...
[catboost] trial 00 COMPLETE loss=0.01859 (348s) best=0.01859
[catboost] trial 01 COMPLETE loss=0.01847 (209s) best=0.01847
[catboost] trial 02 COMPLETE loss=0.01979 (126s) best=0.01847
[catboost] trial 03 COMPLETE loss=0.01846 (679s) best=0.01846
[catboost] trial 04 COMPLETE loss=0.01858 (664s) best=0.01846
[catboost] trial 05 COMPLETE loss=0.01840 (130s) best=0.01840
[catboost] trial 06 COMPLETE loss=0.01860 (130s) best=0.01840
```

```
[catboost] trial 07 COMPLETE loss=0.02076 (693s) best=0.01840
[catboost] trial 08 COMPLETE loss=0.01911 (210s) best=0.01840
[catboost] trial 09 COMPLETE loss=0.01836 (210s) best=0.01836
[catboost] trial 10 COMPLETE loss=0.01961 (212s) best=0.01836
[catboost] trial 11 COMPLETE loss=0.01837 (128s) best=0.01836
[catboost] trial 12 COMPLETE loss=0.01836 (130s) best=0.01836
[catboost] trial 13 COMPLETE loss=0.01837 (130s) best=0.01836
[catboost] trial 14 COMPLETE loss=0.01852 (211s) best=0.01836
[catboost] confirmation trial 12: search fold 0.01836 | confirmation fold 0.01795 (seed std 0.00001)
[catboost] confirmation trial 00 (default): search fold 0.01859 | confirmation fold 0.01810 (seed std 0.00003)
[catboost] confirmation trial 09: search fold 0.01836 | confirmation fold 0.01791 (seed std 0.00002)
[catboost] DECISION: better than default on the confirmation fold: gain 0.00020 >= MIN_GAIN 0.0001, wins 3/3 seeds

--- STARTING TUNING FOR FTTRANSFORMER ---
[ft] Running search for 15 trials...
[ft] trial 00 COMPLETE loss=0.01844 (404s, 24 epochs) best=0.01844
[ft] trial 01 COMPLETE loss=0.01844 (588s, 35 epochs) best=0.01844
[ft] trial 02 COMPLETE loss=0.01906 (423s, 26 epochs) best=0.01844
[ft] trial 03 PRUNED   loss=0.01976 (0s) best=0.01844
[ft] trial 04 PRUNED   loss=0.01843 (0s) best=0.01844
[ft] trial 05 PRUNED   loss=0.01840 (0s) best=0.01844
[ft] trial 06 PRUNED   loss=0.02090 (0s) best=0.01844
[ft] trial 07 PRUNED   loss=0.01863 (0s) best=0.01844
[ft] trial 08 PRUNED   loss=0.01866 (0s) best=0.01844
[ft] trial 09 PRUNED   loss=0.01858 (0s) best=0.01844
[ft] trial 10 PRUNED   loss=0.01876 (0s) best=0.01844
[ft] trial 11 COMPLETE loss=0.01842 (352s, 21 epochs) best=0.01842
[ft] trial 12 PRUNED   loss=0.01867 (0s) best=0.01842
[ft] trial 13 COMPLETE loss=0.01866 (420s, 25 epochs) best=0.01842
[ft] trial 14 COMPLETE loss=0.01855 (353s, 21 epochs) best=0.01842
[ft] WARNING: only 6 completed trials (< 8)
[ft] confirmation trial 11: search fold 0.01842 | confirmation fold 0.01772 (seed std 0.00007)
[ft] confirmation trial 00 (default): search fold 0.01844 | confirmation fold 0.01776 (seed std 0.00007)
[ft] DECISION: keeping default: mean gain +0.00004 < MIN_GAIN 0.0001

=== COMPLETED in 226.9 min ===
        model trials (complete/pruned/failed) default (search/confirm) best (search/confirm) seed wins        mean gain adopted
       ResNet                          16/9/0        0.01867 / 0.01780     0.01828 / 0.01748       3/3 +0.00032 (+1.8%)   TUNED
FTTransformer                           6/9/0        0.01844 / 0.01776     0.01842 / 0.01772       2/3 +0.00004 (+0.2%) default
     CatBoost                          15/0/0        0.01859 / 0.01810     0.01836 / 0.01791       3/3 +0.00020 (+1.1%)   TUNED
  ResNet: better than default on the confirmation fold: gain 0.00032 >= MIN_GAIN 0.0001, wins 3/3 seeds
  CatBoost: better than default on the confirmation fold: gain 0.00020 >= MIN_GAIN 0.0001, wins 3/3 seeds
  FTTransformer: keeping default: mean gain +0.00004 < MIN_GAIN 0.0001

{
    "ResNet": {
        "lr": 0.0026669003721056773,
        "dropout": 0.5828160165372797,
```

```
        "weight_decay": 0.0002661901888489054,
        "hidden_dim": 768,
        "batch_size": 512
    },
    "FTTransformer": {
        "lr": 0.001,
        "dropout": 0.2,
        "weight_decay": 1e-05,
        "d_token": 64,
        "n_blocks": 2,
        "attention_heads": 4,
        "batch_size": 64
    },
    "CatBoost": {
        "depth": 4,
        "l2_leaf_reg": 55.99002805118824,
        "learning_rate": 0.10338380617159558,
        "subsample": 0.8285511060504747,
        "iterations": 250
    }
}
```